

2Dヒストグラム・TFile

2026/06/30 ROOT講習会 第3回

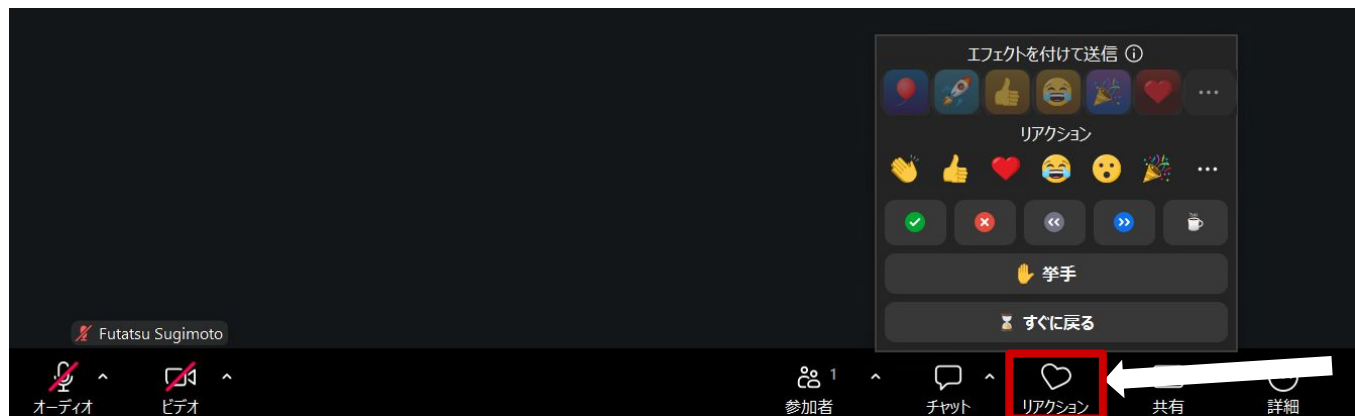
前置き

- ・前回・前々回と同様に初学者向けの想定です。

物足りない方は資料等を見てずんずん進めてください

- ・いつでも質問を受けつけます。挙手とかリアクションをお願いします

(zoom下部のコントロールパネルのリアクション → 挙手とか拍手とか)



これ

自己紹介スライド

杉本 布達（すぎもと ふたつ）

東京大学宇宙線研究所 D1 / ALPACAグループ

ALPACA: ボリビアでやっているsub-PeVガンマ線空気シャワー観測実験



おさらい

- ・第一回ではCLIの使い方について学びました
- ・第二回ではROOTフレームワークで使うC++について学びました

データの扱い方 (C++の型)

int (整数値) / double (実数) / char (文字) / ポインタ (メモリアドレス)

条件で動かし方を変えるもの (C++の制御構文)

if / for / while / break ... などなど

ROOTのヒストグラムTH1D や キャンバス TCanvasなど

今回の位置づけ

- ・今回は第3回として、2次元のヒストグラム・ファイル保存の話をしてします。
 - ・2次元のヒストグラム: 1次元の拡張版です
 - ・ファイル保存: ちょっと毛色の違う話です
 - ・概ねヒストグラム+グラフへのフィッティング(以降の回)ができれば十分？
-
- ・2週間空いていることも考慮して、少しずつ復習を入れるつもりです。

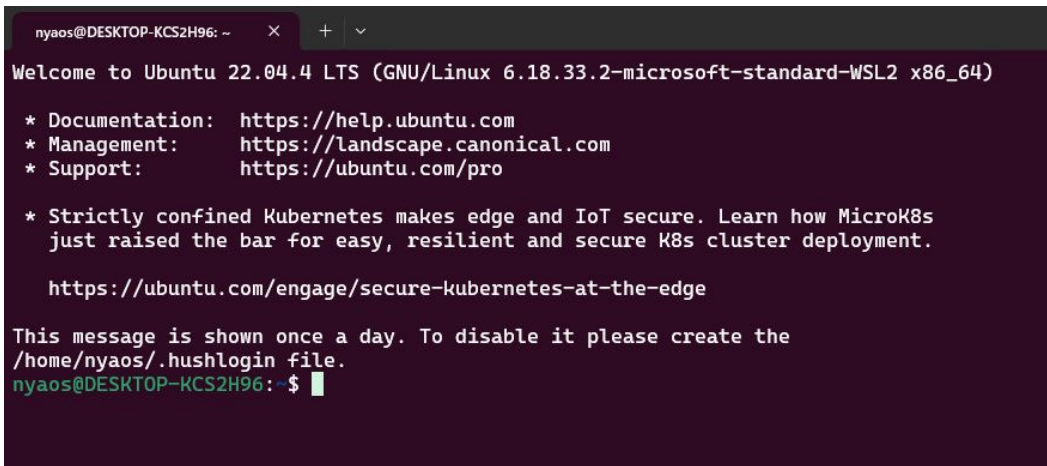
おさらい

- ・まずROOTを実行してみましょう。

おさらい

- ・まずROOTを実行してみましょう。
- ・CLI (コマンドラインインターフェース)を起動します。

例えばWSL (windows subsystem for linux) だとこんな感じです。

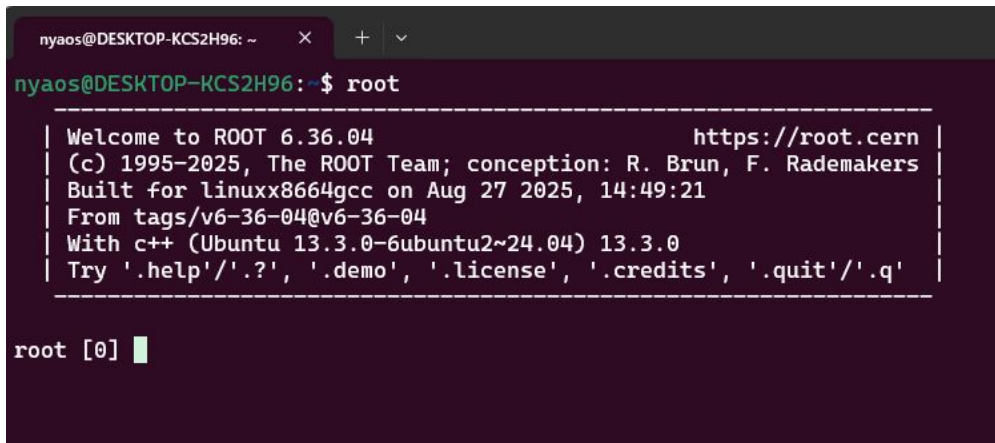


```
nyaos@DESKTOP-KCS2H96: ~  
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
  just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
This message is shown once a day. To disable it please create the  
/home/nyaos/.hushlogin file.  
nyaos@DESKTOP-KCS2H96:~$
```

おさらい

- ・とりあえずROOTを実行してみましょう。
- ・CLI (コマンドラインインターフェース)を起動し、rootと打ち込んでみます。

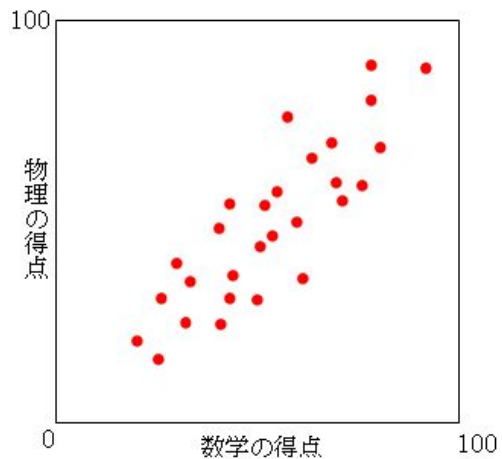
例えばWSL (windows subsystem for linux) だとこんな感じです。

A terminal window titled 'nyaos@DESKTOP-KCS2H96: ~' with a dark background. The prompt is 'nyaos@DESKTOP-KCS2H96:~\$'. The user has entered 'root', which has started the ROOT 6.36.04 application. The application displays a welcome message and version information, enclosed in a dashed box. Below the box, the prompt is 'root [0]'.

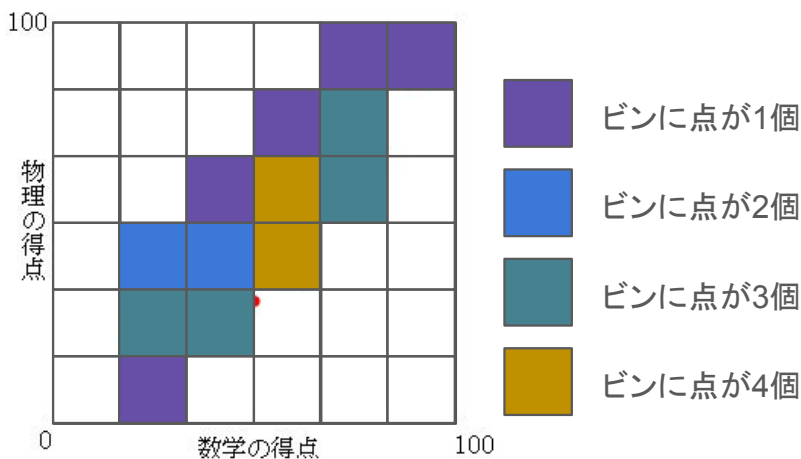
```
nyaos@DESKTOP-KCS2H96: ~  
nyaos@DESKTOP-KCS2H96:~$ root  
-----  
| Welcome to ROOT 6.36.04                                     https://root.cern  
| (c) 1995-2025, The ROOT Team; conception: R. Brun, F. Rademakers  
| Built for linuxx8664gcc on Aug 27 2025, 14:49:21  
| From tags/v6-36-04@v6-36-04  
| With c++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0  
| Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'.'  
|-----  
root [0]
```


2Dヒストグラム

- ・1Dヒストグラムの拡張版です。
- ・いわゆる散布図をグリッドごとに描くことに近いです。



Wikipediaより



人力で2Dヒストグラムにした

2Dヒストグラムを使う理由

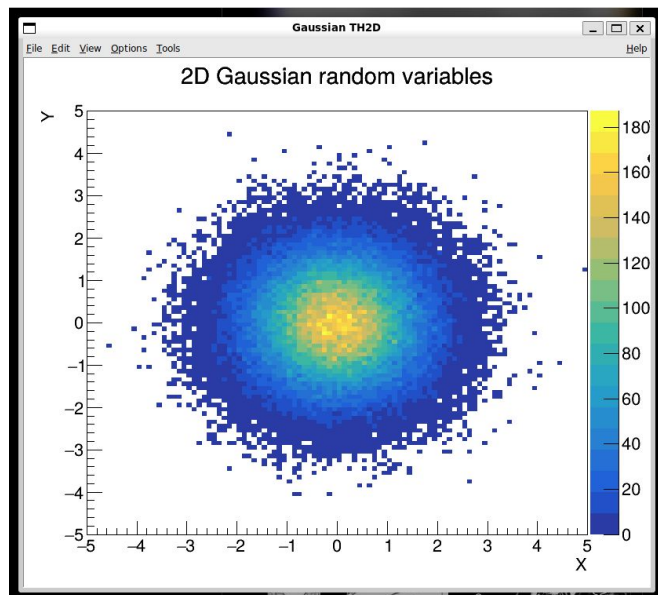
- ・散布図＝ある変数とある変数の相関を見るためのグラフ
- ・ただしどれくらいの値がどこに集中しているかが分かりにくい
 - ビン分けすることで値が集中しているビンを把握することができます
- ・2Dヒストグラムは変数と変数の相関をビンに詰めて把握するために使う
- ・散布図そのものの話は以降の回でフィッティングとともに出てくると思います

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

まずは random_th2d.cxx をダウンロードして、実行してみてください。

単純に2変数の独立なガウス分布に従う乱数を振って表示するマクロです。



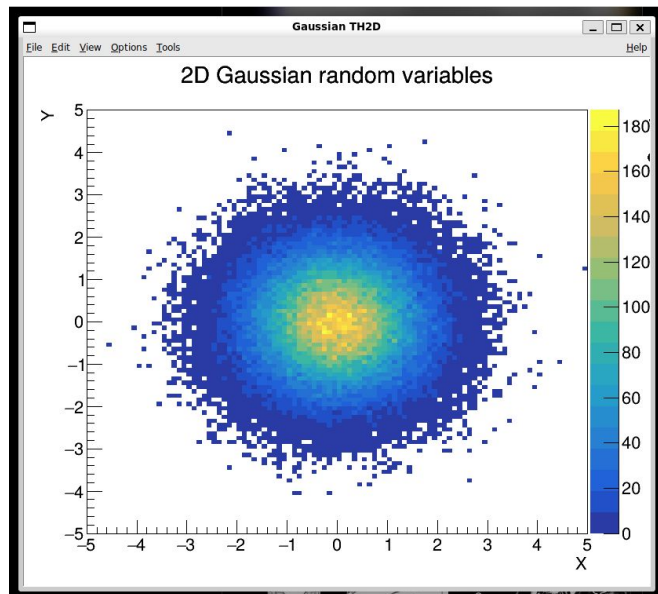
こんな感じの画面が出るはずです。

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

まずは random_th2d.cxx をダウンロードして、実行してみてください。

実行は root random_th2d.cxx で行うのです。



こんな感じの画面が出るはずです。

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

前回のおさらいですが、ヒストグラムを利用するためには次のような手順を踏んでいました。

```
TH1D *th = ...
```

```
th->Fill(...)
```

```
th->Draw()
```

データの構造を決める文を書く

条件を変えたりしてデータを詰める

描画のディテール指定・描画

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

1次元のヒストグラムも、2次元のヒストグラムもこの手順は変わりません。

使うデータの構造が変わるだけです (TH1D -> TH2D)

TH2D *th = ...

データの構造を決める文を書く

th->Fill(...)

条件を変えたりしてデータを詰める

th->Draw()

描画のディテール指定・描画

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

それでは中身を参照していきましょう。

このマクロでまず大事なのは、8 ~ 12行目の次の文です。

なんとなく見覚えがあるような？

```
// TH2D作成
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)
TH2D *h2 = new TH2D(
    "h2",
    "2D Gaussian random variables;X;Y;Counts",
    100, -5.0, 5.0,
    100, -5.0, 5.0
);
```

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

それでは中身を参照していきましょう。

このマクロでまず大事なものは、8～12行目の次の文です。

実は前回使ったヒストグラムと同じような文になっています！

ポインター型

```
// TH2D作成
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)
TH2D *h2 = new TH2D(
    "h2",
    "2D Gaussian random variables;X;Y;Counts",
    100, -5.0, 5.0,
    100, -5.0, 5.0
);
```

等号の左側はポインターである＝メモリを参照していることを示します

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

それでは中身を参照していきましょう。

このマクロでまず大事なのは、8～12行目の次の文です。

実は前回使ったヒストグラムと同じような文になっています！

```
// TH2D作成
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)
TH2D *h2 = new TH2D(
    "h2",
    "2D Gaussian random variables;X;Y;Counts",
    100, -5.0, 5.0,
    100, -5.0, 5.0
);
```

データ型
名前

タイトル;X軸タイトル;Y軸 タイトル

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

それでは中身を参照していきましょう。

このマクロでまず大事なのは、8～12行目の次の文です。

実は前回使ったヒストグラムと同じような文になっています！

```
// TH2D作成
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)
TH2D *h2 = new TH2D(
    "h2",
    "2D Gaussian random variables;X;Y;Counts",
    100, -5.0, 5.0,
    100, -5.0, 5.0
);
```

データ型
名前

タイトル;X軸タイトル;Y軸 タイトル
ビン数・下限・上限(X軸)
ビン数・下限・上限(Y軸)

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

それでは中身を参照していきましょう。

このマクロでまず大事なのは、8～12行目の次の文です。

実は前回使ったヒストグラムと同じような文になっています！

```
// TH2D作成
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)
TH2D *h2 = new TH2D(
    "h2",
    "2D Gaussian random variables;X;Y;Counts",
    100, -5.0, 5.0,
    100, -5.0, 5.0
);
```

データ型

名前

タイトル;X軸タイトル;Y軸
タイトル

ビン数・下限・上限(X軸)

ビン数・下限・上限(Y軸)

等号の右側はどういうヒストグラムにするかを指定しています

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

1Dのヒストグラムるときと比較してみます。

TH1D (前回)

```
TH1D *h1 = new TH1D(  
    "h1",  
    "Gaussian random variable;X;Counts",  
    100, -5.0, 5.0  
);
```

データ型

名前

タイトル;X軸タイトル

ビン数・下限・上限(X軸)

TH2D (今回)

```
// TH2D作成  
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)  
TH2D *h2 = new TH2D(  
    "h2",  
    "2D Gaussian random variables;X;Y;Counts",  
    100, -5.0, 5.0,  
    100, -5.0, 5.0  
);
```

データ型

名前

タイトル;X軸タイトル;Y軸タイトル

ビン数・下限・上限(X軸)

ビン数・下限・上限(Y軸)

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

1Dのヒストグラムるときと比較してみます。

TH1D (前回)

```
TH1D *h1 = new TH1D(  
    "h1",  
    "Gaussian random variable;X;Counts",  
    100, -5.0, 5.0  
);
```

データ型

名前

タイトル;X軸タイトル

ビン数・下限・上限(X軸)

TH2D (今回)

```
// TH2D作成  
// TH2D(name, title, xbins, xmin, xmax, ybins, ymin, ymax)  
TH2D *h2 = new TH2D(  
    "h2",  
    "2D Gaussian random variables;X;Y;Counts",  
    100, -5.0, 5.0,  
    100, -5.0, 5.0  
);
```

データ型

名前

タイトル;X軸タイトル;Y軸タイトル

ビン数・下限・上限(X軸)

ビン数・下限・上限(Y軸)

構文はほぼ同じで、Y軸分のビン数・下限・上限が足されただけ

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

先ほどのTH2D ... というのがデータの構造を決める部分でした。

続けてデータを詰めていきましょう。

TH2D *th = ...

データの構造を決める文を書く

th->Fill(...)

条件を変えたりしてデータを詰める

th->Draw()

描画のディテール指定・描画

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

先ほどのTH2D ... というのがデータの構造を決める部分でした。

続けてデータを詰めていきましょう。以下がデータ生成～詰める部分です。

```
// 乱数生成器
TRandom3 rng(0); // 0なら時刻ベースの seed

// 正規分布に従う乱数を詰める
const int nEvents = 100000;

for (int i = 0; i < nEvents; ++i) {
    double x = rng.Gaus(0.0, 1.0); // mean=0, sigma=1
    double y = rng.Gaus(0.0, 1.0); // mean=0, sigma=1

    h2->Fill(x, y);
}
```

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

先ほどのTH2D ... というのがデータの構造を決める部分でした。

続けてデータを詰めていきましょう。以下がデータ生成～詰める部分です。

まず生成部分を見ていきましょう

```
// 乱数生成器
```

```
TRandom3 rng(0); // 0なら時刻ベースの seed
```

乱数生成器です。

```
// 正規分布に従う乱数を詰める
```

```
const int nEvents = 100000;
```

生成するイベント数

```
for (int i = 0; i < nEvents; ++i) {
```

```
    double x = rng.Gaus(0.0, 1.0); // mean=0, sigma=1
```

```
    double y = rng.Gaus(0.0, 1.0); // mean=0, sigma=1
```

ガウス分布として
乱数を振ります

```
    h2->Fill(x, y);
```

```
}
```


2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

先ほどのTH2D ... というのがデータの構造を決める部分でした。

続けてデータを詰めていきましょう。

前回と同様、データを詰めるのはFill... を使えばよいのでした

```
// 乱数生成器
TRandom3 rng(0); // 0なら時刻ベースの seed

// 正規分布に従う乱数を詰める
const int nEvents = 100000;

for (int i = 0; i < nEvents; ++i) {
    double x = rng.Gaus(0.0, 1.0); // mean=0, sigma=1
    double y = rng.Gaus(0.0, 1.0); // mean=0, sigma=1
    h2->Fill(x, y);
}
```

ここでFill(x, y)をしてデータを詰めます

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

ここまででデータ構造を指定し、データを生成し、詰めてきました。

最後にデータを描画します。

TH2D *th = ...

データの構造を決める文を書く

th->Fill(...)

条件を変えたりしてデータを詰める

th->Draw()

描画のディテール指定・描画

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

データ描画をする部分です。

とりあえずTCanvasは無視してください。

```
// Canvas作成
TCanvas *c1 = new TCanvas("c1", "Gaussian TH2D", 800, 700);

// 見た目調整
gStyle->SetOptStat(0);
h2->Draw("COLZ");

c1->Update();
```

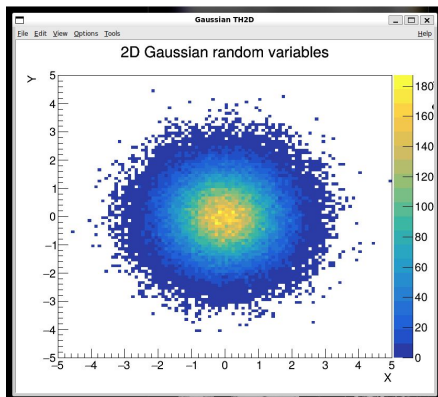
統計情報を出さないようにします

各x,yごとのbinの情報を色で表します

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

改めて最初に行ったマクロの結果を見てみましょう。



当たり前ですが、平均 $x = 0$ に多くのデータ(黄色)があり、 $x = \pm 3$ 付近はデータが少ない(青色)になっていることが分かります

```
// Canvas作成
TCanvas *c1 = new TCanvas("c1", "Gaussian TH2D", 800, 700);

// 見た目調整
gStyle->SetOptStat(0);
h2->Draw("COLZ");

c1->Update();
```

統計情報を出さないようにします

各x,yごとのbinの情報を色で表します

2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

ということで、以下の3つのステップで2Dヒストグラムを描くことができます

TH2D *th = ...

データの構造を決める文を書く

th->Fill(...)

条件を変えたりしてデータを詰める

th->Draw()

描画のディテール指定・描画

【発展】2Dヒストグラムの使い方

ROOTのドキュメントに目を通してみましょう。

<https://root.cern.ch/doc/master/classTH2D.html>

Public Member Functions

TH2D ()

Constructor.

TH2D (const char *name, const char *title, Int_t nbinsx, const Double_t *xbins, Int_t nbinsy, const Double_t *ybins)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, const Double_t *xbins, Int_t nbinsy, Double_t ylow, Double_t yup)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, const Float_t *xbins, Int_t nbinsy, const Float_t *ybins)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, Double_t xlow, Double_t xup, Int_t nbinsy, const Double_t *ybins)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, Double_t xlow, Double_t xup, Int_t nbinsy, Double_t ylow, Double_t yup)

Constructor (see **TH2::TH2** for explanation of parameters)

【発展】2Dヒストグラムの使い方

ROOTのドキュメントに目を通してみましょう。

<https://root.cern.ch/doc/master/classTH2D.html>

TH2D (**const** char *name, **const** char *title, **Int_t** nbinsx, **const** **Double_t** *xbins, **Int_t** nbinsy, **Double_t** ylow, **Double_t** yup)
Constructor (see **TH2::TH2** for explanation of parameters)



今回用いたクラス(ヒストグラムを作る文)の情報が載っています！

【発展】2Dヒストグラムの使い方

ROOTのドキュメントに目を通してみましょう。

<https://root.cern.ch/doc/master/classTH2D.html>

そのほかにも既存の配列を使うなどの方法も書かれています

Public Member Functions

TH2D ()

Constructor.

TH2D (const char *name, const char *title, Int_t nbinsx, const Double_t *xbins, Int_t nbinsy, const Double_t *ybins)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, const Double_t *xbins, Int_t nbinsy, Double_t ylow, Double_t yhigh, Double_t yup)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, const Float_t *xbins, Int_t nbinsy, const Float_t *ybins)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, Double_t xlow, Double_t xup, Int_t nbinsy, const Double_t *ybins)

Constructor (see **TH2::TH2** for explanation of parameters)

TH2D (const char *name, const char *title, Int_t nbinsx, Double_t xlow, Double_t xup, Int_t nbinsy, Double_t ylow, Double_t yhigh, Double_t yup)

Constructor (see **TH2::TH2** for explanation of parameters)



【演習】2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

では、少しマクロをいじってみましょう。

1. まず、ビンの数をx軸、y軸ともに100 -> 40まで減らしてみましょう。

ビンの数はTH2Dを作る際に指定するのです。

2. 描画するときのDraw()に書く文字を”COLZ”から”SURF2”に変えてみましょう

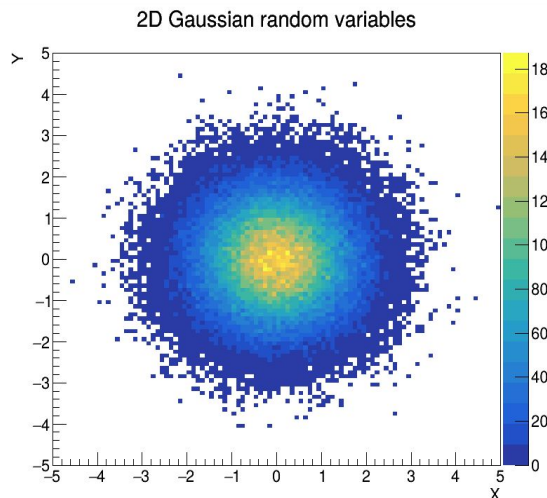
描画は最後にしていたのです。

【演習】2Dヒストグラム の 使い方

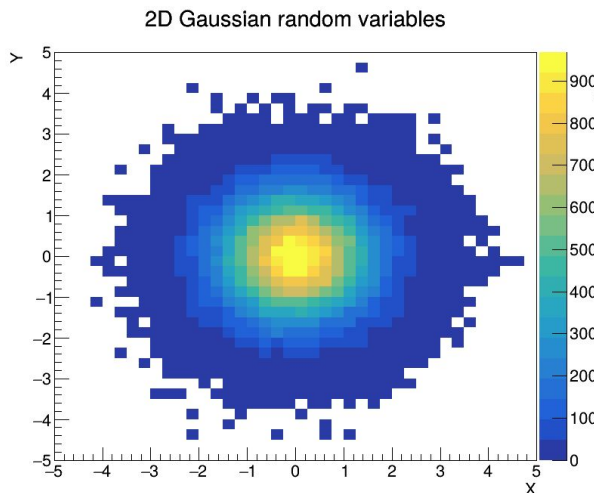
使用するマクロ: random_th2d.cxx

答えです。描画のされ方は環境に依存するかもしれません。

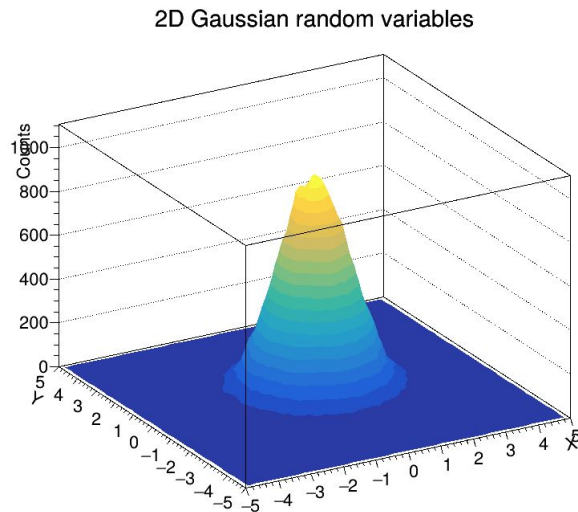
元マクロ



1.ビン数を100→40に減らした後



2.描画時に”SURF2”を指定



【演習】2Dヒストグラムの使い方

使用するマクロ: random_th2d.cxx

答え: random_th2d_ans.cxx

答えです。描画のされ方は環境に依存するかもしれません。

1.ビン数を100→40に減らす

```
// TH2D作成
// TH2D(name, title, xbins, xmin
TH2D *h2 = new TH2D(
    "h2",
    "2D Gaussian random variable
    //previous 100, -5.0, 5.0,
    //previous 100, -5.0, 5.0,
    40, -5.0, 5.0, //answer
    40, -5.0, 5.0 //answer
);
```

赤枠の100, と書かれている部分がビン数
なのでした。100を40に変更します。

2.描画時に”SURF2”を指定

```
// previous h2->Draw("COLZ");
h2->Draw("SURF2"); // answer
```

描画するときに”COLZ”と書いていましたが、それ
を変更すると別の形状で表示されます。

質問タイム

一旦休憩です。

質問があれば受け付けます。

TFile

今まではデータを処理して表示するという話をしていました。

ここからはデータを処理して保存するという話をします。

データ解析では、データ処理をして保存したい場面が出てくると思います。

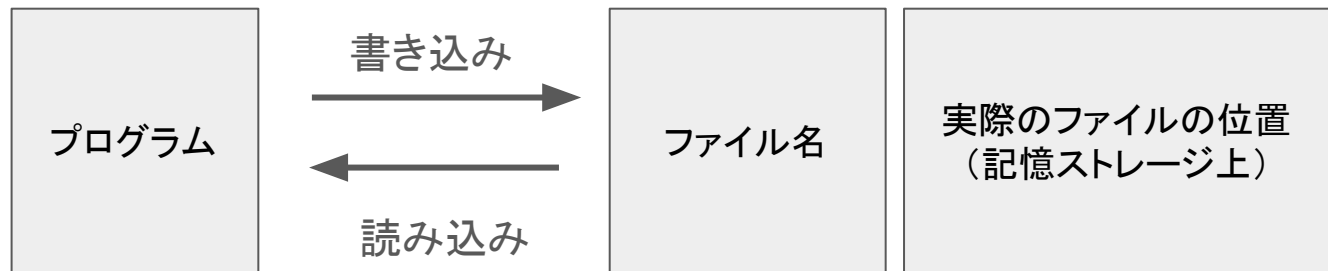
そのようなときに出てくるのがTFileです。

コンピューターに何かを保存する

まず、コンピューター上でのファイルの読み込み・書き込みの話をします。

完全に理解する必要はありません。

概ね次のような形です。



大事ななのは、プログラムで「ファイル名・読みたい / 書きたい」を指定することです

TFile

使用するマクロ: tfile_hist.cxx

では、TFileで具体的にファイルの読み書きをどうやって実現するのでしょうか？

次のような手順になります。

書き読み共通

```
TFile *f = new TFile("filename", "READ"  
/ "RECREATE")
```

ファイルの開き方を決める

書く場合

```
th->Write();
```

ヒストグラムか何かを書き出す

読む場合

```
fin->Get("objectname");
```

ヒストグラムか何かを読み出す

TFile

使用するマクロ: tfile_hist.cxx

まずファイルへの書き込みを見ていきましょう。

ファイルの読み書きは「ファイル名・読みたい / 書きたい」が大事なのでした。

そこで、まず書き込み先を作る文を見えます。

```
TFile *fout = new TFile("hist.root", "RECREATE");
```

TFileというクラスの
ポインター

ファイル名
(rootで作る場合は
慣例で.rootという
拡張子にします)

書くときの
オプション

TFile

使用するマクロ: tfile_hist.cxx

まずファイルへの書き込みを見ていきましょう。

今回はROOTファイルを作成、データを書き込み、読み出すマクロを使います。

マクロの流れとしては下の通りです。

ファイルを書き込み用を開く

ヒストグラムを書き出す

ファイルを読み出し用を開く

ヒストグラムを読み出す

TFile

使用するマクロ: tfile_hist.cxx

まずファイルへの書き込みを見ていきましょう。

その後、書き込むものを作ります。今回は1次元のヒストグラムです。

1Dヒストグラムを作る文

```
TH1D *h_write = new TH1D("h_gaus", "random 1d histogram;x;Events", 100, 0, 10);
```

```
for (int i = 0; i < 10000; i++) {  
    h_write->Fill(gRandom->Gaus(5.0, 1.0));  
}
```

ヒストグラムにデータを詰める文

```
h_write->Write();
```

TFile

使用するマクロ: tfile_hist.cxx

まずファイルへの書き込みを見ていきましょう。

最後に、ファイルへの書き込みの指示を行います。

```
TH1D *h_write = new TH1D("h_gaus", "random 1d histogram;x;Events", 100, 0, 10);  
  
for (int i = 0; i < 10000; i++) {  
    h_write->Fill(gRandom->Gaus(5.0, 1.0));  
}
```

```
h_write->Write();
```

ファイルへの書き込み指示

TFile

使用するマクロ: tfile_hist.cxx

流れをざっと復習しましょう。

```
TFile *fout = new TFile("hist.root", "RECREATE");
```

ファイルを書き込み用に開く

```
TH1D *h_write = new TH1D("h_gaus", "random 1d histogram;x;Events", 100, 0, 10);  
  
for (int i = 0; i < 10000; i++) {  
    h_write->Fill(gRandom->Gaus(5.0, 1.0));  
}
```

ヒストグラムを作ってデータを詰める

```
h_write->Write();
```

ファイルに書き込む指示を出す

TFile

使用するマクロ: tfile_hist.cxx

これで書き出しについては学びました。次は読み出しです。

読むファイルを開く構文も書くファイルを開く構文と似ており、違いはオプション指定になります。

オプションを"RECREATE"から"READ"に変更するだけです。

```
TFile *fin = new TFile("hist.root", "READ");
```

TFileというクラスの
ポインター

ファイル名

読むときの
オプション

TFile

使用するマクロ: tfile_hist.cxx

開いた後は、もともとファイルに書いたデータと同じデータ構造を作る必要があります。

もともとは1次元のヒストグラム (TH1D)だったので、同じ構造で読み出します。

読み出すのは file->Get("object name")という形式です。

TH1Dというクラスの
ポインター

ファイルの中に入っている
ヒストグラムの名前

```
TH1D *h_read = (TH1D*)fin->Get("h_gaus");
```

```
h_read->Draw();
```

TFile

使用するマクロ: tfile_hist.cxx

書き込み時のヒストグラムの文に戻ってみます。

```
TH1D *h_write = new TH1D("h_gaus", "random 1d histogram;x;Events", 100, 0, 10);
```

TH1Dというクラスの
ポインター

書き込み時のヒストグラムの名
前と対応

```
TH1D *h_read = (TH1D*)fin->Get("h_gaus");
```

```
h_read->Draw();
```

ファイルの中に入っている
ヒストグラムの名前

TFile

使用するマクロ: tfile_hist.cxx

ファイル名自体は分かるけど、ヒストグラムの名前が分からない場合は別手法で知ることができます。

ターミナルで `root -l (ファイルの名前)` と打って、`.ls` と打ちます。

正しくファイルに書き込みができていれば、ヒストグラムの名前とタイトルが出るはず

```
nyaos@DESKTOP-KCS2H96:~$ root -l hist.root
root [0]
Attaching file hist.root as _file0...
(TFile *) 0x5e5175c72920
root [1] .ls
TFile**      hist.root
TFile*       hist.root
KEY: TH1D    h_gaus;1      random 1d histogram
```



ヒストグラムの名前 / タイトル

TFile

使用するマクロ: tfile_hist.cxx

ということで、ざっとTFileへのデータ書き込み・読み込みの話をしました。

次回以降にヒストグラムではなくデータを書き込む方法の話もあると思います。

書き読み共通

```
TFile *f = new TFile("filename", "READ"  
/ "RECREATE")
```

ファイルの開き方を決める

書く場合

```
th->Write();
```

ヒストグラムか何かを書き出す

読む場合

```
fin->Get("objectname");
```

ヒストグラムか何かを読み出す

【演習】TFile

使用するマクロ: tfile_hist.cxx

簡単にですが、最後に少し手を動かしましょう。

rootファイルに書き込み・読み込みしたあとに描画するヒストグラムと、元のヒストグラムが同じであることを確認してみましょう。

ヒント: プログラムの一部はダブルスラッシュ(//)を前につけると無効化できます

【演習】TFile

使用するマクロ: tfile_hist.cxx

簡単にですが、最後に少し手を動かしましょう。

rootファイルに書き込み・読み込みしたあとに描画するヒストグラムと、元のヒストグラムが同じであることを確認してみましょう。

ヒント: プログラムの一部はダブルスラッシュ(//)を前につけると無効化できます

ヒント2: ヒストグラムはDraw()をつけると描画できるのでした。

【演習】TFile

使用するマクロ: tfile_hist.cxx

簡単にですが、最後に少し手を動かしましょう。

rootファイルに書き込み・読み込みしたあとに描画するヒストグラムと、元のヒストグラムが同じであることを確認してみましょう。

このような形で、書き込み前のヒストグラムを描画し、他の動作をスキップ

```
TRandom3 rng(42);
for (int i = 0; i < 10000; i++) {
    h_write->Fill(rng.Gaus(5.0, 1.0));
}
h_write->Draw();
//h_write->Write();
//fout->Close();

//TFile *fin = new TFile("hist.root", "READ");
//TH1D *h_read = (TH1D*)fin->Get("h_gaus");

//h_read->Draw();
```

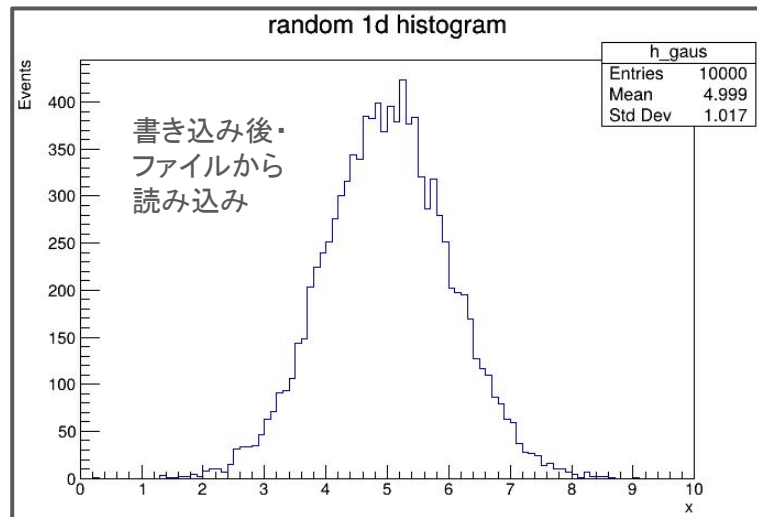
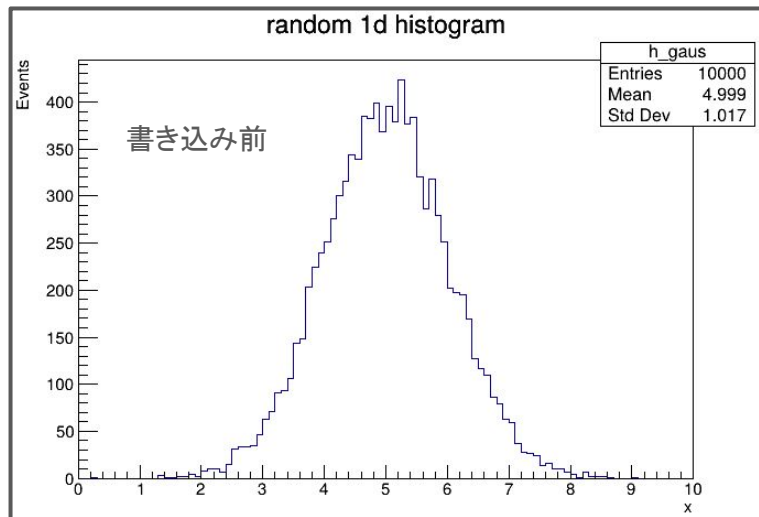
}

【演習】TFile

使用するマクロ: tfile_hist.cxx

簡単にですが、最後に少し手を動かしましょう。

rootファイルに書き込み・読み込みしたあとに描画するヒストグラムと、元のヒストグラムが同じであることを確認してみましょう。



まとめ

本日はざっとTH2DとTFileへのデータ書き込み・読み込みの話をしました。

次回以降にヒストグラムではなくデータを書き込む方法の話もあると思います。

うまくAIなどを使いながらROOTに親しむ一助にいただけると幸いです！

また、YMAPへの参加も歓迎しています！

参加はこちらから: <http://www.icrr.u-tokyo.ac.jp/YMAP/join.html>